

NETWORKS AND PROTOCOLS

Lab 06 - Wireshark/Scapy - UDP/TCP

User Datagram Protocol (UDP)

In this lab, we will explore the basics of the UDP transport protocol. By now, you should have developed strong skills in using Wireshark. Therefore, we won't outline the steps as explicitly as we did in earlier labs, nor will we include example screenshots for every step.

Objectif

- Understanding how UDP and TCP work.
- Learn how to send and receive messages via UDP and TCP.
- Observe the characteristics of connectionless transmission (packet loss, reception order).

Task 1: Capture UDP packets using Wireshark

Step1: Start capturing

Start capturing packets in Wireshark and let it run without performing any specific actions. You will notice that some UDP packets sent or received by other devices will appear in your capture.

Once you stop the packet capture, apply a filter in Wireshark to display only the UDP packets sent and received by your host, you can use `udp` or `ip.proto == 17` in filter text. Select one of these UDP packets and expand the UDP fields in the details pane.

- What are the different source and destination IP addresses ?

.....
.....

- What are the different source and destination ports ?

.....
.....

- What is the length of the UDP segment captured ?

.....

- Are you able to read the content ? Is it the same think for all other UDP segments ?

.....

.....

Task 2: Client and server Python UDP example

In this task, we will create UDP segments by setting up a simple client-server communication to exchange messages. Using Wireshark, we will capture and analyse the packets exchanged, as well as the connectionless transmission characteristics.

Step1: Creating a UDP server

- Write a program that listens on a specific UDP port.
- Display the messages received and send a simple acknowledgement to the client.

```

1 import socket
2
3 server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
4 server_socket.bind(("0.0.0.0", 12345))
5
6 print("UDP server waiting for messages...")
7
8 while True:
9     data, address = server_socket.recvfrom(1024)
10    print(f"Message received : {data.decode()} de {address}")
11    response = f"Message received : {data.decode()}"
12    server_socket.sendto(response.encode(), address)

```

Step 2: Creating a UDP client

- Write a program that sends messages to the server.
- Display the server's response.

```

1 import socket
2
3 client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
4
5 server_address = ("127.0.0.1", 12345)
6
7 while True:
8     message = input("Enter a message to send : ")
9     client_socket.sendto(message.encode(), server_address)
10    response, _ = client_socket.recvfrom(1024)
11    print(f"Server response : {response.decode()}")

```

Step 3: Launch Wireshark and capture traffic

- Launch Wireshark and select the loopback, figure 1. It will be displayed all the all packets sent and received in 127.0.0.1.
- capture the udp segments, use udp or ip.proto==17 filter, figure 2.

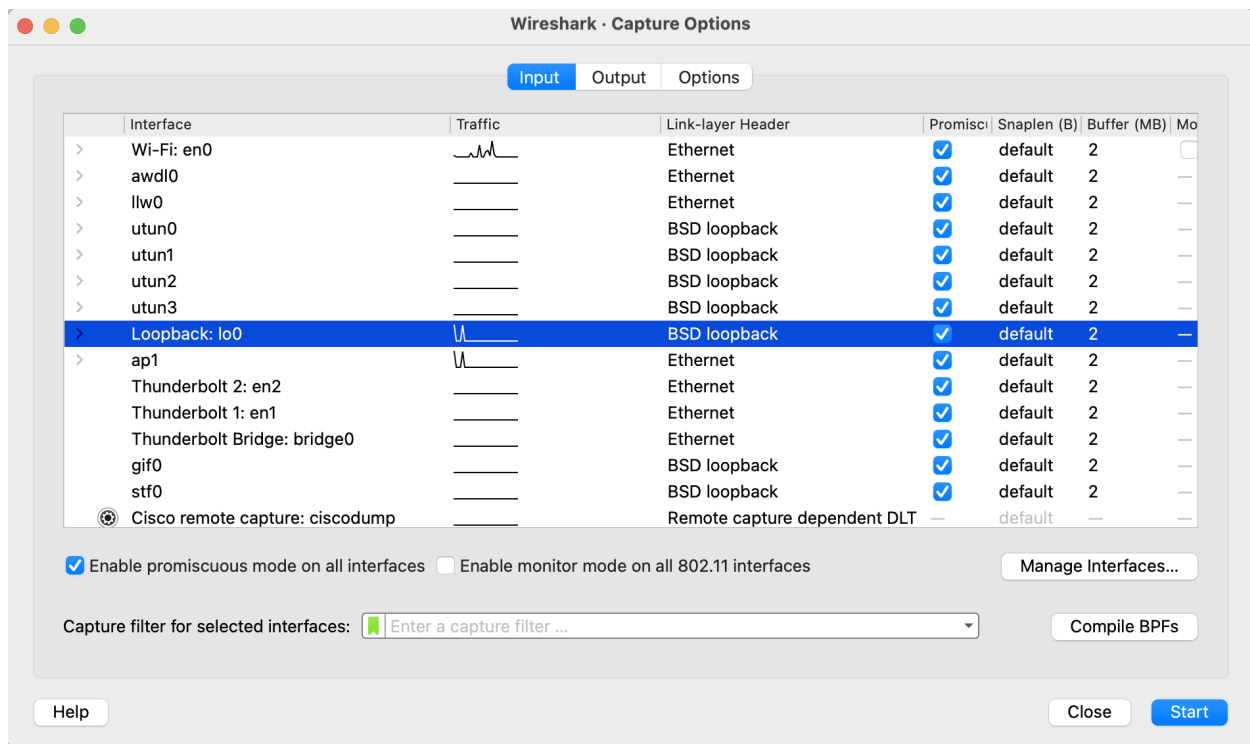


Figure 1: Selecting the loopback interface

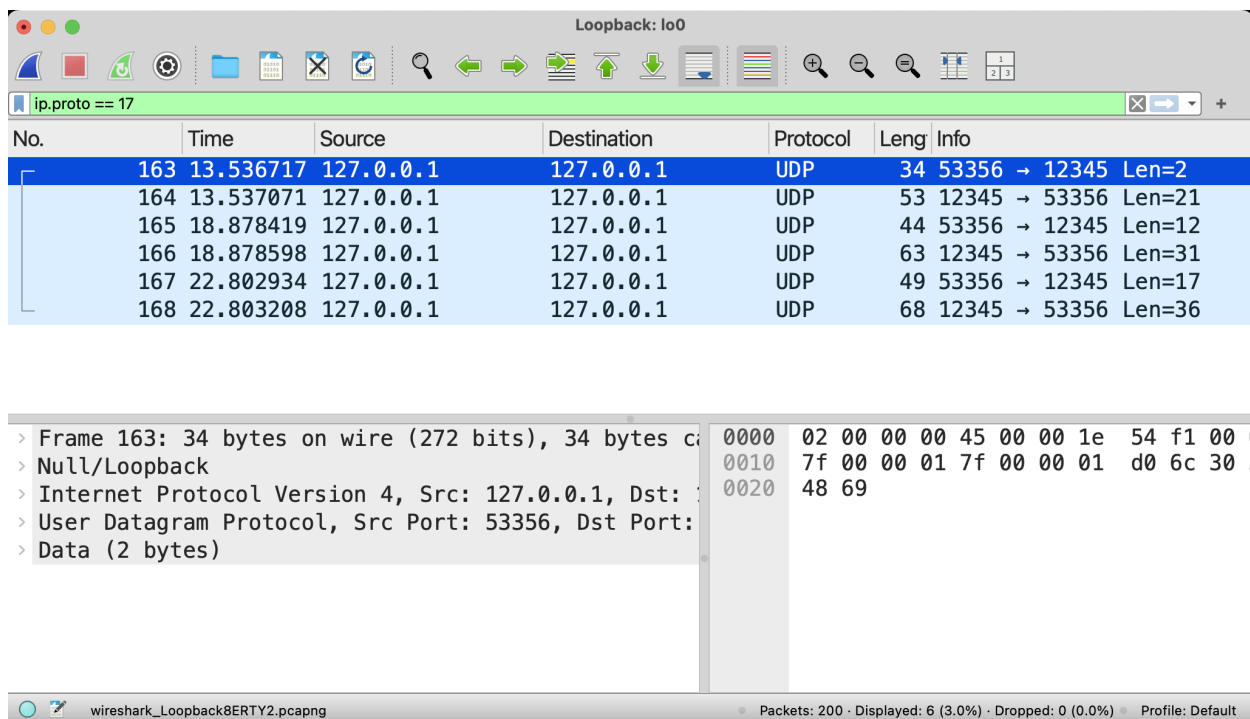


Figure 2: Packets captured and filtered

Step 4: Analyse prackets captured

- Analyse the content of the packets captured, could you see the messages sent and received ?

Task 3: Generate an RTP traffic

In this task we will generate rtp traffic encapsulated in the udp using a Google Meet conference, then analyse the resulting traffic with Wireshark.

Step 1: Filter packets

- Start Wireshark and capture packets using the `rtp` filter

Step 2: Launch a meet session

- Use Google Meet to create and start an instant meeting.
- Analyse traffic using Wireshark, figure 3.

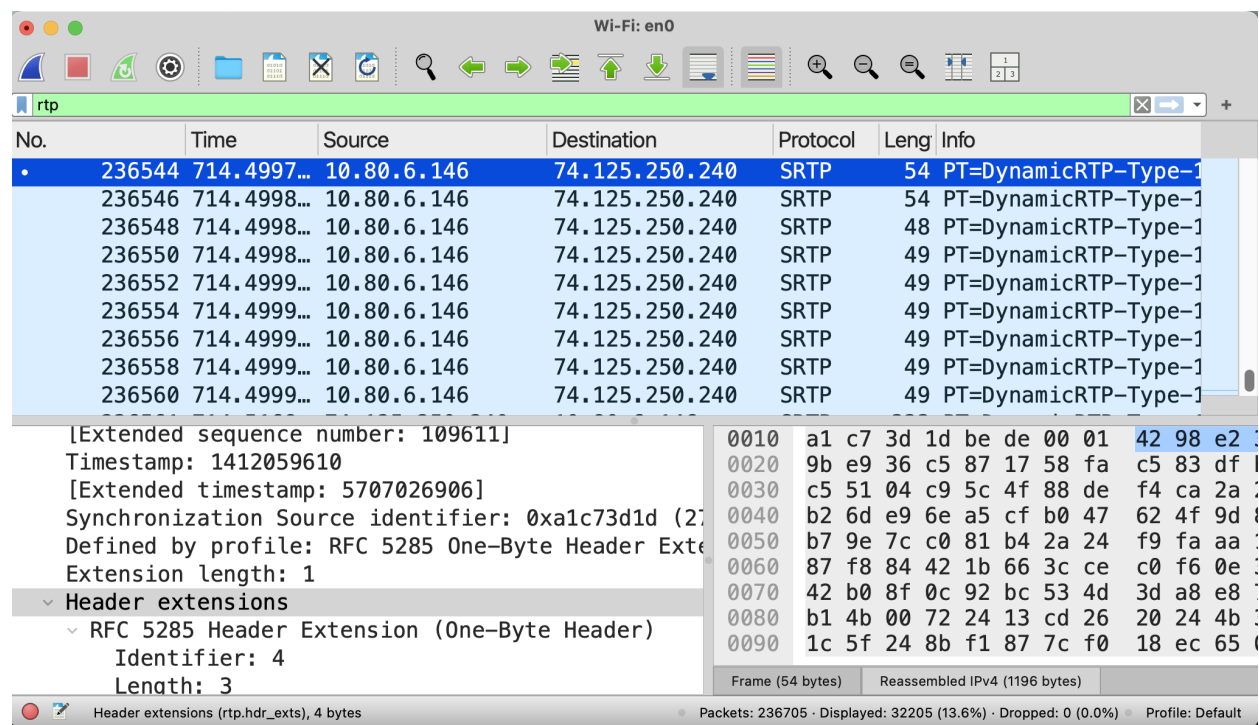


Figure 3: RTP traffic captured

- Analyse the fields of the received rtp packets
- Go through the packets received one by one, what did you notice about the Sequence Number, Timestamp and Synchronisation Source Identifier (SSID)?

.....

.....

.....

Task 4: Generate UDP segment using scapy

Step 1: Creating and sending a simple UDP packet

- Create a script to send a UDP packet to a specific destination.

```
1 from scapy.all import *
2
3 packet = IP(dst="192.168.100.16") / UDP(dport=12345) / "Hello UDP" #Update the IP address
4 print("Packet generated :")
5 packet.show()
6
7 send(packet)
```

Step 2: Receiving and capturing UDP responses

- We will also listen for responses on a specific port.

```
1 from scapy.all import *
2
3 print("Waiting for UDP responses...")
4 sniff(filter="udp and port 12345", prn=lambda x: x.show(), count=5)
```

- Capture incoming UDP packets.
- Analyse their content.

Step 3: Generating an incorrect checksum

- Construct UDP packets with anomalies (such as an incorrect checksum or an invalid size) to observe network behaviour.

```
1 from scapy.all import *
2
3 packet = IP(dst="192.168.100.16") / UDP(dport=12345, sport=54321, chksum=0xFFFF) / "Invalid
   checksum" #Update the IP address
4 print("Packet with incorrect checksum :")
5 packet.show()
6
7 send(packet)
```

- Observe the recipient's reactions.
- Capture traffic with Wireshark to analyse anomalies.

Step 4: Simulation of a UDP scanner

- Implement a scanner to detect open UDP ports on a target machine.

```
1 from scapy.all import *
2
3 target_ip = "192.168.100.16"
4 ports = [53, 67, 123, 500] # Commonly used UDP ports
5
6 for port in ports:
7     packet = IP(dst=target_ip) / UDP(dport=port)
8     response = sr1(packet, timeout=1, verbose=0)
9     if response:
10         print(f"Response received on port {port}")
11     else:
12         print(f"No response on port {port}")
```

Extra exercise

In this extra exercise, we're going to use Scapy to create a DNS server. The client will send a UDP request to resolve a domain name, and the server will respond with a fictitious IP address, all using Scapy to generate and manipulate the packets.

Step 1: Creating the UDP server with Scapy

The server listens on a specific port, processes the requests received and replies to the client with a personalised UDP response.

```
1 from scapy.all import *
2
3 def handle_request(packet):
4     if UDP in packet and packet[UDP].dport == 5353: # Port DNS fictif
5         print(f"Request received : {packet[Raw].load.decode()}")
6         # Creating the UDP answer
7         response = IP(dst=packet[IP].src , src=packet[IP].dst) / \
8                 UDP(dport=packet[UDP].sport , sport=5353) / \
9                 f"192.168.1.100"
10        print("Response sent : 192.168.1.100")
11        send(response)
12
13 print("Server waiting...")
14 sniff(filter="udp and port 5353" , prn=handle_request)
```

Step 2: Creating the UDP client with Scapy

The client sends a UDP request containing a domain name to the server.

```
1 from scapy.all import *
2
3 server_ip = "127.0.0.1"
4 server_port = 5353
5
6 domain_name = "www.example.com"
7
8 packet = IP(dst=server_ip) / UDP(dport=server_port , sport=12345) / domain_name
9 print("Request sent :")
10 packet.show()
11
12 response = sr1(packet , timeout=2)
13 if response:
14     print(f"Answer received : {response[Raw].load.decode()}")
15 else:
16     print("No reply received.")
```

Transmission Control Protocol (TCP)

Task 1: TCP using a local web server

To understand how the TCP protocol works, we are going to use a local web server like XAMPP to explore the TCP protocol in the context of HTTP (which is based on TCP).

Important: XAMPP must be installed and running

Step 1: Prepare the html file

- The `html` file must be saved in the `htdocs` folder (or any root folder on your web server).
- Add a simple HTML file to test :

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4     <meta charset="UTF-8">
5     <title>XAMPP Test</title>
6 </head>
7 <body>
8     <h1>Welcome to my local server</h1>
9     <p>This is a test page.</p>
10 </body>
11 </html>
```

Step 2: Analysis of HTTP/TCP exchanges

- Start Wireshark, select the loopback interface and apply a filter on `tcp.port == 80` or `tcp.port == 443`.
- Load the `http://localhost/<link to the page>` html file into your browser.
- Identify the associated TCP exchanges (handshake, send data, close).
- What is the value of ISN?
.....
- What is the window size?
.....
- What is the size of the TCP segments sent?
.....
- How are packets numbered (sequence and acknowledgement numbers)?
.....

Step 3: Modify web files to observe effects on TCP

1. Testing a large file:

- Add a larger file to `htdocs` (for example, an image or an HTML file of several kilobytes).
- Load this file into the browser and observe how TCP segments the data.

2. Introduction to a simple PHP script:

- Add a `test.php` file to `htdocs` with the following content:

```
1 <?php
2 echo "Hello , this is a PHP script running on the local server.";
3 ?>
```

- Load `https://localhost/<link to the php file>` into your browser.
- Use Wireshark to identify the differences between loading a static file (HTML) and loading a dynamically generated file (PHP).

3. Simulate high latency:

- Add an artificial delay to the PHP script to simulate a slow response:

```
1 <?php
2 sleep(5);
3 echo "Hello after a delay.";
4 ?>
```

- Look in Wireshark to see how TCP handles late responses.

Task 2: TCP client/server in Python

We are going to write a TCP client/server in Python, introduce specific conditions (simulated packet losses, delays), and observe the impact on exchanges with Wireshark.

Step 1: Implementing a TCP client and server

1. Server script:

- Start a TCP server that listens on a port and responds to each message sent by a client.

```
1 import socket
2
3 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4 server.bind(('0.0.0.0', 12345))
5 server.listen(5)
6 print("Server ready, waiting for connections...")
7 client_socket, addr = server.accept()
8 print(f"Connection established with {addr}")
9
10 while True:
11     data = client_socket.recv(1024)
12     print(f"Data received : {data.decode('utf-8')}")
13     client_socket.sendall(b"Received !")
14 client_socket.close()
```

2. Client script:

- Implement a client that sends several messages to the server.

```
1 import socket
2 import time
3
4 client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5 client.connect(('127.0.0.1', 12345))
6 for i in range(5):
7     message = f"Message {i+1}".encode('utf-8')
8     client.sendall(message)
9     response = client.recv(1024)
10    print(f"Server response : {response.decode('utf-8')}")
11    time.sleep(1) # Pause between each transmission
12 client.close()
```

3. Test and capture:

- Start the server and the client. Check that they are exchanging messages correctly and capture the various segments using Wireshark.

Step 2: Simulation of network conditions

1. Study of TCP retransmissions:

- **Modification:** Introduce an artificial delay in the server to simulate a slowdown.

```
1 import time
2 time.sleep(5) # Pause for 5 seconds before answering
```

- **Observation:** Launch the client and use Wireshark to observe the retransmissions caused by the delay.

2. Packet loss simulation:

- **Modification:** Introduce a condition where the server ignores certain messages :

```
1 if "2" in data.decode('utf-8'):
2     continue # Ignores messages containing "2"
```

- **Observation:** Analyse the retransmissions in Wireshark.

3. Variation in data size:

- **Modification:** Send a larger message to observe the breakdown into TCP segments.

```
1 message = b"A" * 2000 # Message of 2000 bytes
```

Task 3: exploring TCP with Scapy

In this new task, we're going to explore the features of the TCP protocol using scapy.

Step 1: Creating a manual TCP session

1. Perform a full TCP handshake

- Creating the SYN

```

1 from scapy.all import *
2
3 target_ip = "192.168.100.1" # Replace with the target IP address
4 target_port = 80           # Destination Port
5 syn_packet = IP(dst=target_ip)/TCP(dport=target_port , flags="S")
6 syn_response = sr1(syn_packet)
7 print(syn_response.summary())

```

- SYN-ACK -> Sending an ACK

```

1 ack_packet = IP(dst=target_ip)/TCP(dport=target_port , flags="A" , seq=syn_response.ack ,
    ack=syn_response.seq + 1)
2 send(ack_packet)
3 print("Handshake completed.")

```

2. Data transfer with customised packets

- After the handshake, send a TCP packet with data :

```

1 data_packet = IP(dst=target_ip)/TCP(dport=target_port , flags="PA" , seq=syn_response.
    ack , ack=syn_response.seq + 1)/"Hello Server !"
2 send(data_packet)

```

- Using Wireshark, capture and identify the server response. Does it conform to the TCP protocol?

- **Questions:**

- What happens if you change the sequence number manually?

.....

3. Closing the TCP session

- Close the session properly

```

1 fin_packet = IP(dst=target_ip)/TCP(dport=target_port , flags="FA" , seq=syn_response.ack
    , ack=syn_response.seq + 1)
2 fin_response = sr1(fin_packet)
3 ack_close_packet = IP(dst=target_ip)/TCP(dport=target_port , flags="A" , seq=
    fin_response.ack , ack=fin_response.seq + 1)
4 send(ack_close_packet)

```

- **Questions:**

- What happens if the client sends a packet after the session has been closed?

.....

- **Questions :**

- * What happens if the client sends a packet after the session has been closed?

.....

- * Which TCP flags are involved in the closure?

.....

Step 2: Advanced scenarios and error simulation

1. Simulate packet loss

- Send a SYN packet but do not reply with an ACK :

```
1 send(IP(dst=target_ip)/TCP(dport=target_port , flags="S"))
2 # No ACK sent
```

- Questions :

- What does the server do in this case? (Analyse in Wireshark).

.....

- How long before the server stops retransmitting?

.....

2. Injection of malformed packets

- Send a packet with a false sequence number:

```
1 bad_packet = IP(dst=target_ip)/TCP(dport=target_port , flags="PA" , seq=9999, ack=1)/"
  Fake packet !"
2 send(bad_packet)
```

- Questions:

- How does the server react? (Look in Wireshark).

.....

- Which TCP mechanisms detect this packet as incorrect?

.....